FUEL DELIVERY SYSTEM IN AZURE

GE23432 SOFTWARE CONSTRUCTION

Submitted by

ABISHEK D - 240701013

II YEAR B.E

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

RAJLAKSHMI ENGINEERING COLLEGE
THANDALAM, CHENNAI 602 105

MAY 2026

CS23432 – SOFTWARE CONSTRUCTION

EXPERIMENT – 1
STUDY OF AZURE

Aim

To study the fundamentals of Microsoft Azure, its architecture, services, and how it supports cloud computing for building, deploying, and managing applications.

Azure

Microsoft Azure is a cloud computing platform developed by Microsoft that provides a wide range of services such as computing, storage, networking, databases, analytics, and artificial intelligence. It enables users to access computing resources over the internet instead of relying on local hardware.

Azure supports different types of cloud models:

- Public Cloud – services are available over the internet
- Private Cloud – used by a single organization
- Hybrid Cloud – combination of both

Azure also offers three main service models:

- Infrastructure as a Service (IaaS) – provides virtual machines, storage, and networking
- Platform as a Service (PaaS) – provides platforms for developing and deploying applications
- Software as a Service (SaaS) – delivers software applications over the internet

Key Components of Azure

- Virtual Machines (VMs): Provide scalable computing power
- Storage Services: Store data in blobs, files, or disks
- Azure Networking: Enables secure communication between resources
- Azure App Services: Used to build and host web applications
- Azure SQL Database: Managed database service
- Azure Monitor: Tracks performance and health of resources

Features of Azure

- Scalability: Resources can be increased or decreased based on demand
- Reliability: High availability through global data centers
- Security: Built-in security and compliance features
- Cost Efficiency: Pay-as-you-go pricing model
- Global Reach: Services available across multiple regions worldwide

Applications of Azure

- Web and mobile app development
- Data storage and backup
- Machine learning and AI applications
- Internet of Things (IoT) solutions
- Big data analytics

Result

The study of Microsoft Azure shows that it is a powerful cloud platform that provides scalable, secure, and cost-effective solutions for deploying and managing applications. It simplifies infrastructure management and supports modern technologies like IoT, big data, and web development.

CS23432 – SOFTWARE CONSTRUCTION

EXPERIMENT – 2
FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS

Aim

To study and analyze the Functional Requirements (FR) and Non-Functional Requirements (NFR) of the Fuel Delivery System for managing fuel orders, delivery tracking, and payment processing.

Description

The Fuel Delivery System is a smart application developed to allow customers to order fuel online and receive doorstep delivery. The system manages fuel types such as petrol, diesel, and LPG, and handles order placement, real-time delivery tracking, and secure online payment. The system uses location-based services to assign the nearest delivery agent and notify the customer about the delivery status. The Functional Requirements define the major functionalities of the system including user registration, fuel ordering, agent assignment, delivery tracking, and invoice generation. The Non-Functional Requirements describe the quality attributes of the system such as performance, scalability, security, usability, and reliability.

Functional Requirements

- The system shall allow users to register and log in using their credentials.
- The system shall allow users to select fuel type (Petrol, Diesel, LPG) and quantity.
- The system shall allow users to place fuel delivery orders with delivery address.
- The system shall assign the nearest available delivery agent automatically.
- The system shall provide real-time GPS tracking of the delivery agent.
- The system shall send notifications to users regarding order status updates.
- The system shall process payments through multiple payment gateways.
- The system shall generate a digital invoice after successful delivery.
- The system shall allow admins to manage fuel stock and delivery agents.

- The system shall store all order history and delivery records in the database.

Non-Functional Requirements

- The system should process order placement and confirmation within 3–5 seconds.
- The system should support multiple concurrent users through cloud-based deployment.
- The system should ensure 99.9% uptime for uninterrupted service availability.
- User data and payment information should be securely transmitted using encryption.
- The interface should be simple and intuitive for users with minimal technical knowledge.

Result

Thus, the Functional Requirements and Non-Functional Requirements of the Fuel Delivery System were studied successfully. The system requirements ensure accurate fuel order management, real-time delivery tracking, secure payment handling, and user-friendly performance for effective fuel delivery services.

CS23432 – SOFTWARE CONSTRUCTION

EXPERIMENT – 3
CREATE PROJECT IN AZURE DEVOPS

Aim

To create a project named "Fuel Delivery System" in Azure DevOps for managing software development activities using Git version control and Agile methodology.

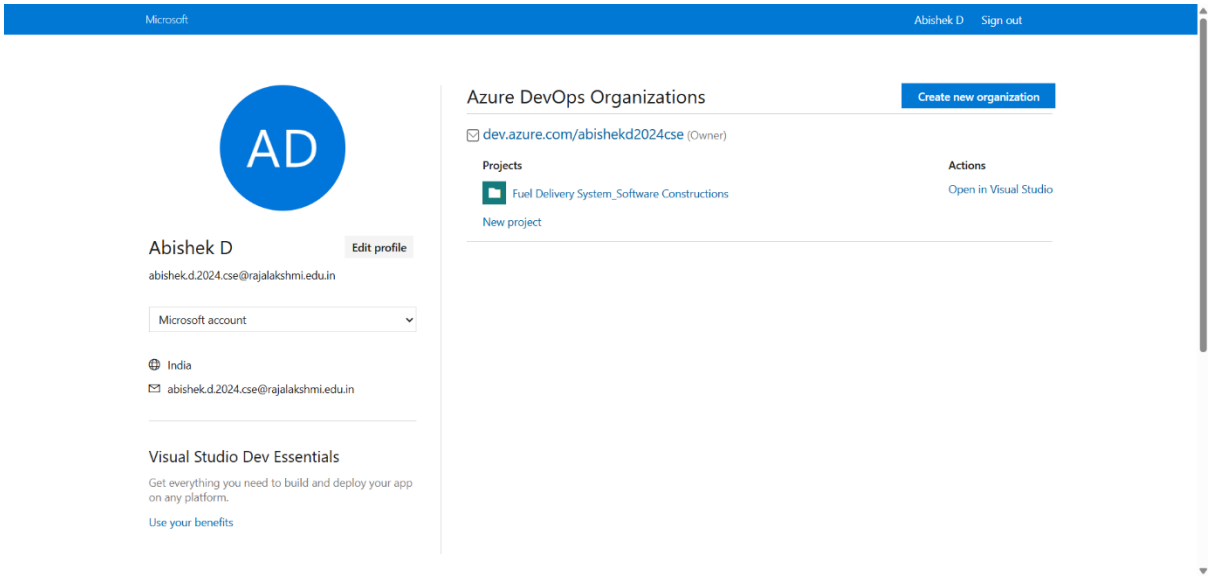
Description

The "Fuel Delivery System" project is developed using Azure DevOps to manage and organize software development activities effectively. The project focuses on developing a smart application that allows customers to order fuel online and receive real-time delivery tracking and payment services. Using Azure DevOps, the project enables efficient planning, task management, version control, and team collaboration throughout the software development lifecycle. The platform supports Agile methodology for managing work items such as epics, user stories, tasks, and bugs. It also provides Git repositories for source code management and tracking project progress. The project includes functionalities such as fuel ordering, agent assignment, GPS tracking, payment processing, and invoice generation. Azure DevOps helps in maintaining project workflow, improving collaboration among team members, and ensuring systematic software development and deployment.

Steps to Create the "Fuel Delivery System" Project in Azure DevOps

- Open a web browser on the system.
- Visit the official Azure DevOps website: https://dev.azure.com
- Sign in using a Microsoft account.
- Click on the "New Project" option available on the dashboard.
- Enter the project name as "Fuel Delivery System".
- Provide the project description explaining the fuel delivery application.
- Select the project visibility as Public or Private.
- Choose Git as the version control system.
- Select Agile as the work item process.
- Click on the "Create" button.
- The "Fuel Delivery System" project dashboard will be created successfully.

- Configure repositories, boards, and pipelines for project management and development activities.



Result

Thus, the "Fuel Delivery System" project was successfully created in Azure DevOps with Git version control and Agile process for effective software development and project management.

CS23432 – SOFTWARE CONSTRUCTION

EXPERIMENT – 4
EPIC, THEME AND USER STORY

Aim

To create and manage Epics, Themes, and User Stories for the "Fuel Delivery System" project in Azure DevOps using Agile project management methodology.

Theme

The theme of the "Fuel Delivery System" project is to develop an intelligent and user-friendly fuel ordering and delivery management system that helps customers order fuel conveniently, track deliveries in real time, and make secure payments. The system is designed to manage fuel types such as Petrol, Diesel, and LPG, assign delivery agents automatically, and generate digital invoices upon delivery completion.

Epics and User Stories for "Fuel Delivery System"

Epic 1: User Authentication and Profile Management

User Story:

As a user, I want to register and log in to my account so that I can securely access my fuel orders and delivery history.

Epic 2: Fuel Order Placement

User Story:

As a user, I want to select fuel type and quantity and place an order with my delivery address so that I can receive fuel at my doorstep.

Epic 3: Delivery Agent Assignment and Tracking

User Story:

As a user, I want the system to assign the nearest delivery agent and allow me to track the delivery in real time using GPS so that I know the estimated arrival time.

Epic 4: Payment and Invoice Generation

User Story:

As a user, I want to pay for my fuel order securely using online payment methods and receive a digital invoice so that I can maintain payment records.

Epic 5: Admin Panel and Stock Management

User Story:

As an admin, I want to manage fuel stock levels, monitor delivery agents, and view all order records so that I can ensure smooth and uninterrupted fuel delivery operations.

Result

Thus, the Epics, Themes, and User Stories for the "Fuel Delivery System" project were successfully created in Azure DevOps using Agile methodology for effective requirement analysis and project management.

CS 23432 – SOFTWARE CONSTRUCTION

EXPERIMENT – 5
CREATE USER STORY IN AZURE DEVOPS

Aim

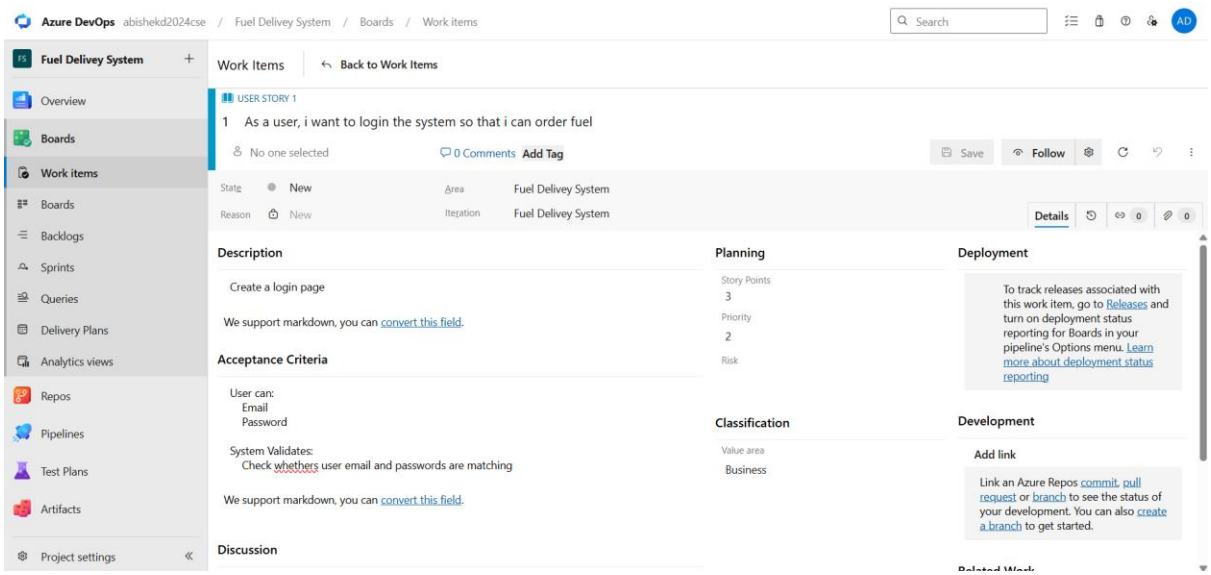
To create and manage a User Story in Azure DevOps for effective project planning, requirement tracking, and agile software development.

Description

Azure DevOps is a cloud-based platform developed by Microsoft that provides tools for software development, project management, testing, and deployment. In Agile methodology, a User Story is used to describe a software feature from the end user's perspective. It helps developers understand user requirements clearly and organize project tasks efficiently. A User Story in Azure DevOps contains details such as the title, description, acceptance criteria, priority, and assigned team member. It enables teams to track progress, manage backlogs, and collaborate effectively during software development. For the Fuel Delivery System project, user stories can be created for functionalities such as fuel ordering, agent assignment, GPS tracking, payment processing, and invoice generation.

Steps to Create a User Story in Azure DevOps

- Open the web browser and go to https://dev.azure.com.
- Sign in using a Microsoft account.
- Create a new project or open the existing "Fuel Delivery System" project.
- In the left-side menu, select Boards.
- Click on Work Items or Backlogs.
- Select New Work Item.
- Choose User Story from the list.
- Enter the title of the user story.
- Example: "As a user, I want to place a fuel delivery order so that I can receive fuel at my doorstep."
- Add a detailed description and acceptance criteria.
- Set priority, iteration, and assign the task to a team member if needed.
- Click Save & Close to store the user story.



Result

Thus, a User Story was successfully created in Azure DevOps for the Fuel Delivery System project. The user story helps in organizing project requirements, tracking development progress, and improving team collaboration in Agile software development.

CS23432 – SOFTWARE CONSTRUCTION

EXPERIMENT – 6
CLASS DIAGRAM

Aim

To design and study the Class Diagram of the Fuel Delivery System using UML concepts in order to represent the relationship between different system components involved in fuel ordering, delivery tracking, and payment processing.

Class Diagram

The class diagram of the Fuel Delivery System illustrates the static structure of the application and shows how different classes interact with each other to perform fuel delivery operations.

The system begins with the User class, where customers can register, log in, and place fuel orders. The User class contains attributes such as userID, name, email, phone, and address, along with methods for registration and login.

The Order class manages fuel orders placed by users. It includes attributes such as orderID, fuelType, quantity, deliveryAddress, orderDate, and status. The Order class is associated with the User class through a one-to-many relationship.

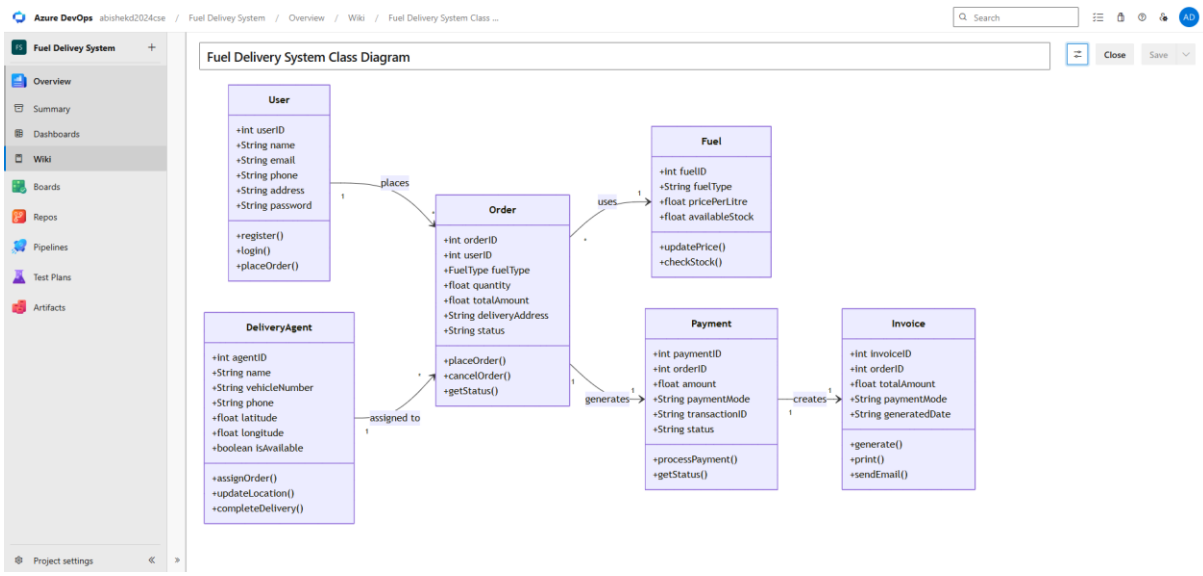
The Fuel class stores information about available fuel types including fuelID, fuelType (Petrol/Diesel/LPG), pricePerLitre, and availableStock. This class is linked to the Order class to retrieve fuel pricing and availability.

The DeliveryAgent class manages information about delivery personnel including agentID, name, vehicleNumber, currentLocation, and availability status. The system assigns the nearest available agent to each fuel order automatically.

The Payment class handles payment transactions including paymentID, amount, paymentMode, transactionID, and paymentStatus. It is associated with the Order class to record and confirm payment for each delivery.

The Invoice class generates digital invoices for completed deliveries, storing invoiceID, orderID, totalAmount, generatedDate, and invoiceStatus. It is linked to both the Order and Payment classes.

The Admin class manages the overall system including fuel stock updates, agent management, and order monitoring. The class diagram also represents relationships such as association, aggregation, and composition between the classes.



Result

Thus, the Class Diagram for the Fuel Delivery System was designed and studied successfully. The diagram clearly represents the classes, attributes, methods, and relationships involved in fuel ordering, agent assignment, payment processing, and invoice generation within the system.

CS23432 – SOFTWARE CONSTRUCTION

EXPERIMENT – 7
SEQUENCE DIAGRAM

Aim

To design and study the Sequence Diagram of the Fuel Delivery System in order to understand the interaction and message flow between different system components during fuel order placement, agent assignment, delivery tracking, and payment processing.

Sequence Diagram

The sequence diagram of the Fuel Delivery System represents the step-by-step interaction between system modules involved in processing fuel orders and completing deliveries. It shows how data flows between the user, web interface, order management module, agent assignment module, payment gateway, and database.

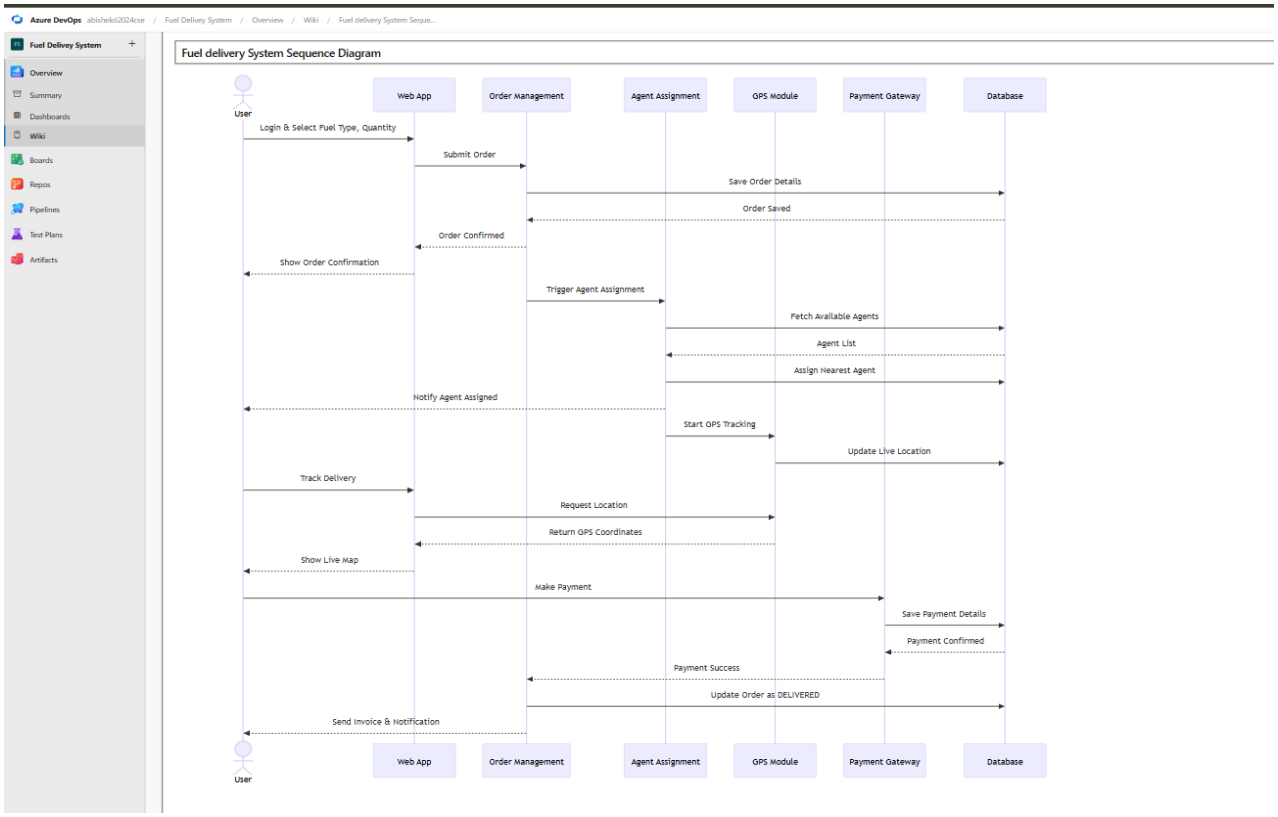
The process begins when the User logs in through the Web Application and selects the fuel type, quantity, and delivery address to place an order. The web application forwards the order details to the Order Management Module, which validates the order and saves it to the Database.

Once the order is confirmed, the Agent Assignment Module is triggered. This module retrieves the list of available delivery agents and their current locations from the database, identifies the nearest agent, and assigns the order to that agent. A notification is sent to both the user and the assigned delivery agent.

During delivery, the GPS Tracking Module continuously updates the delivery agent's live location to the database. The user can monitor the delivery agent's real-time location through the web application at any time.

After successful delivery, the system prompts the user to make payment through the Payment Gateway. The payment gateway processes the transaction and returns the payment status to the Order Management Module. Upon successful payment, the Invoice Generation Module creates and sends a digital invoice to the user.

Finally, the order status is updated as "Delivered" in the database and the user receives a delivery confirmation notification. The sequence diagram clearly illustrates the synchronous communication and interaction among all modules in the Fuel Delivery System.



Result

Thus, the Sequence Diagram for the Fuel Delivery System was designed and studied successfully. The diagram illustrates the sequential interaction between different components involved in fuel order placement, delivery agent assignment, GPS tracking, payment processing, and invoice generation within the system.

CS23432 – SOFTWARE CONSTRUCTION

EXPERIMENT – 8
ER DIAGRAM

Aim

To design and study the Entity Relationship (ER) Diagram for the Fuel Delivery System in order to represent the database structure, entities, attributes, and relationships involved in fuel ordering, agent assignment, and payment processing.

ER Diagram

The Entity Relationship (ER) Diagram of the Fuel Delivery System represents the database design used to store and manage user information, fuel orders, delivery agent details, payment records, and invoices. The ER diagram helps in understanding how data is organized and how different entities are related within the system.

The system begins with the USER entity, which stores details such as userID, name, email, password, phone, and address. A user can place multiple fuel orders, establishing a one-to-many relationship between the USER and ORDER entities.

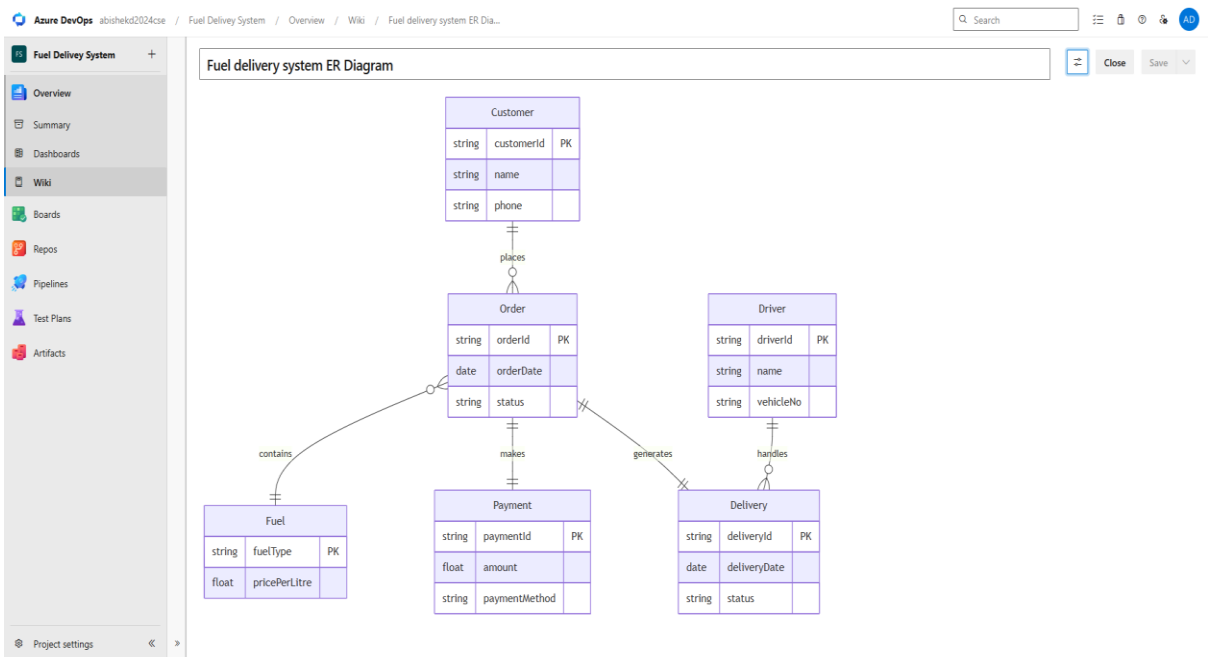
The ORDER entity stores information related to fuel orders, including orderID, fuelType, quantity, deliveryAddress, orderDate, status, and userID as a foreign key. Each order is assigned to a delivery agent.

The FUEL entity stores details about available fuel types including fuelID, fuelType (Petrol/Diesel/LPG), pricePerLitre, and availableStock. The fuelID acts as a foreign key in the ORDER entity.

The DELIVERY_AGENT entity contains attributes such as agentID, agentName, vehicleNumber, phone, currentLocation, and availability. The agent is linked to orders through a one-to-many relationship, as one agent can handle multiple orders.

The PAYMENT entity stores payment transaction information including paymentID, amount, paymentMode, transactionID, paymentStatus, and orderID as a foreign key. Each order has one associated payment record.

The INVOICE entity stores digital invoice details including invoiceID, orderID, paymentID, totalAmount, generatedDate, and invoiceStatus. The ER diagram clearly shows primary keys, foreign keys, and relationships such as one-to-many and one-to-one associations among entities, ensuring proper database organization and data consistency.



Result

Thus, the ER Diagram for the Fuel Delivery System was designed and studied successfully. The diagram represents the entities, attributes, primary keys, foreign keys, and relationships required for efficient storage and management of fuel order, delivery, payment, and invoice data within the system.

CS23432 – SOFTWARE CONSTRUCTION

EXPERIMENT – 9
SYSTEM ARCHITECTURE

AIM

To design and implement a Fuel Delivery System that allows customers to order fuel online, assigns delivery agents automatically, tracks deliveries in real time, processes payments securely, and stores all data using cloud-based architecture.

OBJECTIVE

- To allow users to place fuel orders online with location-based delivery.
- To assign the nearest available delivery agent automatically using GPS.
- To provide real-time tracking of delivery agents during order fulfillment.
- To process payments securely through integrated payment gateways.
- To generate digital invoices and maintain order history in cloud storage.

DESCRIPTION

The Fuel Delivery System is an intelligent web-based application that allows customers to order fuel (Petrol, Diesel, LPG) from their location and receive doorstep delivery. Users place orders through the web application, and the system processes the request using location services and an agent management module. The nearest delivery agent is assigned automatically, and the customer can track the delivery in real time through GPS integration. Payments are processed securely through online payment gateways. All order data, delivery records, and payment information are stored in cloud storage for future access and analysis.

The system architecture consists of the following modules:

- Web Application Module
- Order Management Module
- Agent Assignment Module
- GPS Tracking Module
- Payment Gateway Module
- Invoice Generation Module
- Cloud Data Storage Module

TECHNOLOGIES USED

- C / Python / Node.js
- GPS and Location Services
- HTML / CSS / JavaScript
- Cloud Storage (Azure)
- Payment Gateway API
- SQL / NoSQL Database

WORKING PRINCIPLE

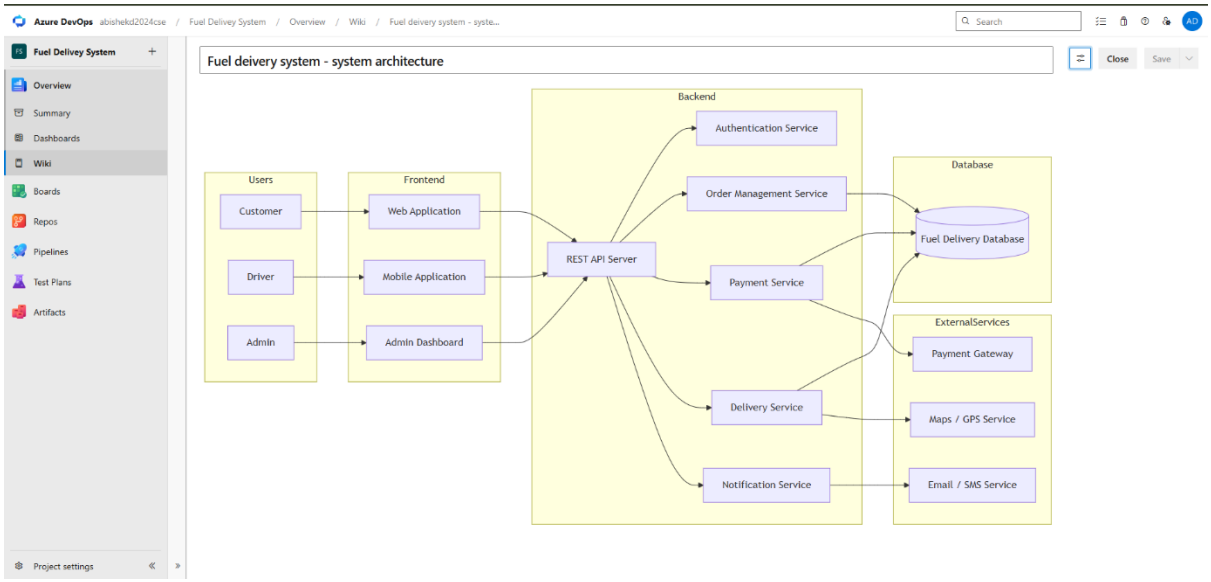
- User registers and logs in to the web application.
- User selects fuel type, quantity, and provides delivery address.
- Order Management Module validates and stores the order.
- Agent Assignment Module selects the nearest available delivery agent.
- Delivery agent is notified and begins delivery.
- GPS Tracking Module updates delivery location in real time.
- Upon delivery, payment is processed through the Payment Gateway.
- Invoice is generated and stored in cloud storage.

ADVANTAGES

- Convenient doorstep fuel delivery
- Automated and optimized agent assignment
- Real-time GPS delivery tracking
- Secure online payment processing
- Cloud-based scalable architecture

LIMITATIONS

- Requires active internet connection
- Delivery limited to serviceable geographic areas
- GPS accuracy may vary in remote locations



RESULT

The Fuel Delivery System successfully manages fuel order placement, automatic agent assignment, real-time GPS tracking, secure payment processing, and digital invoice generation using cloud-based architecture. The system ensures efficient and reliable fuel delivery for all users.

CS23432 – SOFTWARE CONSTRUCTION

EXPERIMENT – 10
BUSINESS ARCHITECTURE

AIM

To design the business architecture of the Fuel Delivery System for managing fuel ordering, agent assignment, real-time delivery tracking, payment processing, and invoice generation using cloud-based services.

OBJECTIVE

- To develop an intelligent fuel delivery workflow for customers.
- To automate delivery agent assignment using GPS and location services.
- To provide real-time delivery tracking for customers.
- To integrate secure payment gateways with order management.
- To improve fuel delivery efficiency and customer satisfaction.

DESCRIPTION

The Fuel Delivery System Business Architecture represents the complete workflow of the system from fuel order placement to delivery completion and invoice generation. The architecture explains how users interact with the web application, how orders are processed using the order management module, and how delivery agents are assigned and tracked using GPS services. The system further processes payments and generates digital invoices based on completed deliveries.

The architecture ensures proper communication between modules such as user layer, application layer, processing layer, data management layer, and output layer to achieve efficient and reliable fuel delivery operations.

The system includes the following layers and modules:

- User Layer
- Web Application Layer
- Order Management Module
- Agent Assignment Module
- GPS Tracking Module
- Payment Gateway Module
- Data Management Layer

- Invoice and Notification Module

The User Layer allows customers to register, log in, place fuel orders, and track deliveries. The Web Application Layer acts as the communication interface between users and the backend processing system. The Order Management Module validates, stores, and updates fuel orders in the database.

The Agent Assignment Module uses GPS and location data to identify and assign the nearest available delivery agent automatically. The GPS Tracking Module provides real-time location updates of the delivery agent to the customer. The Payment Gateway Module handles secure online transactions and updates payment status.

The Data Management Layer handles storage and retrieval of all order, agent, payment, and invoice data efficiently using Azure cloud storage. The Invoice and Notification Module generates digital invoices and sends delivery status notifications to users via SMS or email.

TECHNOLOGIES USED

- C / Python / Node.js
- GPS / Location Services API
- Azure Cloud Storage
- Payment Gateway API
- SQL / NoSQL Database
- Web Application Framework

WORKING PROCESS

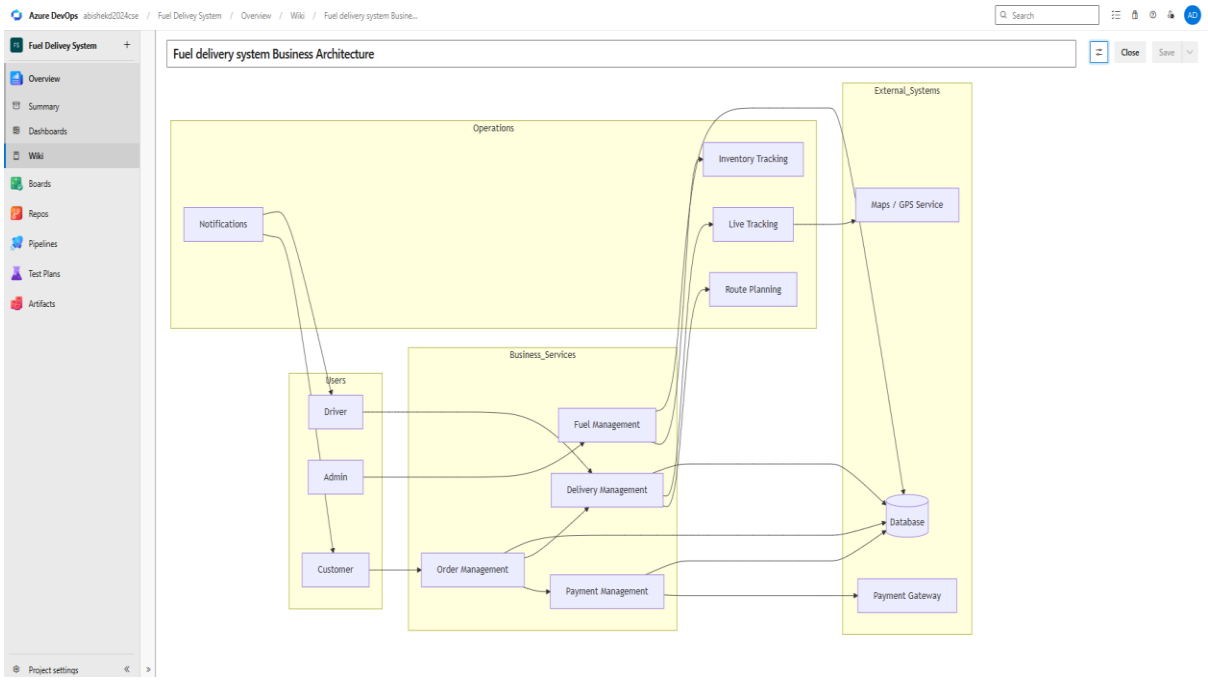
- User places fuel order through the web application.
- Order Management Module validates and stores the order.
- Agent Assignment Module identifies the nearest delivery agent.
- Delivery agent is notified and dispatched.
- GPS Tracking Module updates live delivery location.
- Payment Gateway processes the payment upon delivery.
- Data Management Layer stores all records securely.
- Invoice and Notification Module generates invoice and notifies user.

ADVANTAGES

- End-to-end automated fuel delivery workflow
- Real-time GPS tracking for transparency
- Secure and multi-mode payment support
- Scalable cloud-based data management
- User-friendly business architecture

LIMITATIONS

- Requires stable internet connectivity
- Service coverage limited by agent availability
- GPS accuracy may be reduced in indoor or remote areas



RESULT

The Fuel Delivery System Business Architecture successfully integrates order management, GPS-based agent assignment, real-time delivery tracking, payment processing, and invoice generation into a unified platform. The architecture enables efficient fuel delivery, secure payment handling, and personalized notifications for all users.